

WEEK-9

TASK-1:

To Implement how to List Active Jobs, Display Status Information, Retrieve Job [Ds, Format Output, Update State Information, Test Job Listings

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <string.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;

    pid_t pid;

    char command[50];

    char state[20];
} Job;

Job jobs[MAX_JOBS];

int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
        return;

    jobs[job_count].job_id = job_count + 1;

    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);

    strcpy(jobs[job_count].state, "Running");

    job_count++;
}
```

```

}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );
        if (result > 0)
        {
            if (WIFEXITED(status))
            {
                strcpy(jobs[i].state, "Completed");
            }
            else if (WIFSIGNALED(status))
            {
                strcpy(jobs[i].state, "Terminated");
            }
        }
    }
}

void list_jobs()
{
    update_jobs();
    printf("\n");
    printf("Job ID   PID     Status   Command\n");

```

```

printf("-----\n");
for (int i = 0; i < job_count; i++)
{
    printf("[%6d] %-9d %-12s %s\n",
        jobs[i].job_id,
        jobs[i].pid,
        jobs[i].state,
        jobs[i].command);
}
printf("\n");
}

void get_job(int id)
{
    update_jobs();

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == id)
        {
            printf("\nJob Information\n");
            printf("-----\n");
            printf("Job ID : %d\n", jobs[i].job_id);
            printf("PID   : %d\n", jobs[i].pid);
            printf("Status : %s\n", jobs[i].state);
            printf("Command : %s\n\n", jobs[i].command);
            return;
        }
    }

    printf("Job %d not found.\n", id);
}

```

```

}

void create_job(int seconds, const char *command)
{
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        return;
    }
    if (pid == 0)
    {
        sleep(seconds);
        exit(0);
    }
    add_job(pid, command);
    printf("Created Job [%d] with PID %d\n",
        job_count,
        pid);
}

int main()
{
    int choice;
    int job_id;
    printf("Job Management System\n");
    printf("=====\n");
    while (1)
    {
        printf("\n");
        printf("1. Create Job\n");

```

```
printf("2. List Active Jobs\n");
printf("3. Get Job Information\n");
printf("4. Update Job Status\n");
printf("5. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch (choice)
{
    case 1:
        create_job(10, "sleep 10");
        break;
    case 2:
        list_jobs();
        break;
    case 3:
        printf("Enter Job ID: ");
        scanf("%d", &job_id);
        get_job(job_id);
        break;
    case 4:
        update_jobs();
        printf("Job status updated.\n");
        break;
    case 5:
        printf("Exiting...\n");
        return 0;
    default:
        printf("Invalid choice.\n");
}
```

```

}

return 0;

}

```

```

root@LAPTOP-RJFDBEST:~# nano jobs.c
root@LAPTOP-RJFDBEST:~# ls
File1.txt      copyfile.c      ls_grep          sample.txt
File1.txt*    con_demo.c      ls_grep.c        server
File2.txt      con_demo.c      memory_layout.c  server.c
OSSP           date.txt        multi_pipe        signal_handler
SKILL_Week_4   deadlock.c      multi_pipe.c      signal_handler.c
append.txt     deadlock_prevention.c  normal.txt        source.txt
append_redirect  deadlock_prevention.c  order.txt        stderr_redirect
background_jobs  destination.txt      output.txt        stderr_redirect.c
background_jobs.c  dynamic_memory.c  output_redirect.c  task1.c
builtin_shell    error.txt         path_resolver     task2.c
builtin_shell.c  export_demo.c     path_resolver.c   task3.c
cd_navigation    export_demo.c     pipe_dataflow     task4.c
cd_navigation.c  file.txt          pipeline           task5.c
client           file_copy.c       pipeline.c         task5_old
client.c         filecopy          process            tasks1
combined.txt     filecopy.c        process_demo.c    test.txt
combined_redirect  fork_process      process_exp        test2.txt
command_executor  information.txt    process_states     text.txt
command_executor.c  inode_test        process_states.c   trace.txt
command_history    input.txt          producer_consumer  user.txt
complex_command    input_redirect.c  producer_consumer  variable_expand
complex_command.c  job_table.c        pwd_exit           variable_expand.c
copy              jobs               pwd_exit           waitpid_demo.c
copy.c            jobs               redirect            waitpid_demo.c
'copy.c'          jobs               result.txt          week3
copy1.txt         listing.txt        gcc jobs.c -o jobs
copy_std.txt      root@LAPTOP-RJFDBEST:~#
copyfile          root@LAPTOP-RJFDBEST:~#
copyfile          Job Management System
=====
1. Create Job
2. List Active Jobs
3. Get Job Information
4. Update Job Status
5. Exit
Enter choice: 1

2. List Active Jobs
3. Get Job Information
4. Update Job Status
5. Exit
Enter choice: 3
Enter Job ID: 1

Job Information
-----
Job ID : 1
PID    : 1398
Status : Completed
Command : sleep 10

1. Create Job
2. List Active Jobs
3. Get Job Information
4. Update Job Status
5. Exit
Enter choice: 4
Job status updated.

1. Create Job
2. List Active Jobs
3. Get Job Information
4. Update Job Status
5. Exit
Enter choice: 2

Job ID  PID    Status    Command
-----
[1]    ] 1398    Completed  sleep 10
[2]    ] 1399    Completed  sleep 10

1. Create Job
2. List Active Jobs
3. Get Job Information
4. Update Job Status
5. Exit
Enter choice: 5
Exiting...
root@LAPTOP-RJFDBEST:~# gcc jobs.c -o jobs
root@LAPTOP-RJFDBEST:~#

```

TASK-2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

CODE:

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <signal.h>

#include <sys/wait.h>

#include <errno.h>

int main()
{
    pid_t pid;

    pid_t shell_pgid;

    int status;

    shell_pgid = getpgrp();

    signal(SIGTTOU, SIG_IGN);

    signal(SIGTTIN, SIG_IGN);

```

```

signal(SIGTSTP, SIG_IGN);

printf("Foreground Job Control Demonstration\n");

printf("=====\n");

printf("Shell Process Group ID: %d\n", shell_pgid);

pid = fork();

if (pid < 0)
{
    perror("fork");
    exit(1);
}

if (pid == 0)
{
    setpgid(0, 0);

    printf("\nChild Job Started\n");

    printf("Child PID    : %d\n", getpid());

    printf("Child PGID    : %d\n", getpgrp());

    printf("Child is running in foreground...\n");

    sleep(2);

    printf("Child is stopping itself using SIGSTOP...\n");

    raise(SIGSTOP);

    printf("Child resumed successfully!\n");

    sleep(2);

    printf("Child job completed.\n");

    exit(0);
}

setpgid(pid, pid);

printf("\nTarget Job Identified\n");

printf("-----\n");

printf("Job PID : %d\n", pid);

```

```

printf("Job PGID : %d\n", pid);
printf("\nTransferring terminal control to child...\n");
if (tcsetpgrp(STDIN_FILENO, pid) == -1)
{
    perror("tcsetpgrp");
    return 1;
}
printf("Terminal control transferred to Job %d\n", pid);
if (waitpid(pid, &status, WUNTRACED) == -1)
{
    perror("waitpid");
    return 1;
}
tcsetpgrp(STDIN_FILENO, shell_pgid);
printf("\nParent regained terminal control.\n");
if (WIFSTOPPED(status))
{
    printf("Job State: STOPPED\n");
}
printf("\nResuming Job %d...\n", pid);
tcsetpgrp(STDIN_FILENO, pid);
if (kill(pid, SIGCONT) == -1)
{
    perror("SIGCONT");
    return 1;
}
printf("SIGCONT sent successfully.\n");
if (waitpid(pid, &status, 0) == -1)
{

```

```

        perror("waitpid");

        return 1;
    }

    tcsetpgrp(STDIN_FILENO, shell_pgid);

    printf("\nTerminal control returned to parent.\n");

    if (WIFEXITED(status))
    {
        printf("Job State: COMPLETED\n");

        printf("Exit Status: %d\n", WEXITSTATUS(status));
    }

    else if (WIFSIGNALED(status))
    {
        printf("Job State: TERMINATED\n");
    }

    printf("\nForeground switching test completed successfully.\n");

    return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano foreground_switch.c
root@LAPTOP-RJFD8EBT:~# ls -l foreground_switch.c
-rw-r--r-- 1 root root 3096 Sep 27 16:06 foreground_switch.c
root@LAPTOP-RJFD8EBT:~# gcc foreground_switch.c -o foreground_switch
root@LAPTOP-RJFD8EBT:~# ls -l foreground_switch
-rwxr-xr-x 1 root root 16584 Sep 27 16:07 foreground_switch
root@LAPTOP-RJFD8EBT:~# ./foreground_switch
Foreground Job Control Demonstration
=====
Shell Process Group ID: 1436

Target Job Identified
Child Job Started
-----
Job PID : 1437
Job PGID : 1437

Transferring terminal control to child...
Terminal control transferred to Job 1437
Child PID : 1437
Child PGID : 1437
Child is running in foreground...
Child is stopping itself using SIGSTOP...

Parent regained terminal control.
Job State: STOPPED

Resuming Job 1437...
SIGCONT sent successfully.
Child resumed successfully!
Child job completed.

Terminal control returned to parent.
Job State: COMPLETED
Exit Status: 0

Foreground switching test completed successfully.
root@LAPTOP-RJFD8EBT:~# echo $?
0
root@LAPTOP-RJFD8EBT:~#

```

TASK-3: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
root@LAPTOP-RJFD8EBT:~# nano signal_analysis.c
root@LAPTOP-RJFD8EBT:~# ls -l signal_analysis.c
-rwxr-xr-x 1 root root 3252 Sep 27 16:13 signal_analysis.c
root@LAPTOP-RJFD8EBT:~# cat signal_analysis.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void sigusr1_handler(int sig)
{
    printf("\nSIGUSR1 received by PID %d\n", getpid());
}

void sigint_handler(int sig)
{
    printf("\nSIGINT (Ctrl+C) received by PID %d\n", getpid());
}

int main()
{
    pid_t pid;
    int status;

    printf("Signal Delivery and Process Group Demonstration\n");
    printf("=====\n");

    printf("Parent PID : %d\n", getpid());
    printf("Parent PGID : %d\n", getpgid());

    /* Install signal handlers */
    signal(SIGUSR1, sigusr1_handler);
    signal(SIGINT, sigint_handler);

    /* Create child process */
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid == 0)
    {
        /* Child process */
        setpgid(0, 0);

        printf("\nChild Process\n");
        printf("-----\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PGID : %d\n", getpgid());

        signal(SIGUSR1, sigusr1_handler);
        signal(SIGINT, sigint_handler);

        /* Demonstrate signal mask */
        sigset_t mask;

        sigemptyset(&mask);
        sigaddset(&mask, SIGUSR1);

        printf("\nBlocking SIGUSR1...\n");

        if (sigprocmask(SIG_BLOCK, &mask, NULL) == -1)
        {
            perror("sigprocmask");
            exit(1);
        }

        printf("SIGUSR1 is now blocked.\n");
        printf("Waiting for signal...\n");

        sleep(3);

        printf("\nUnblocking SIGUSR1...\n");

        if (sigprocmask(SIG_UNBLOCK, &mask, NULL) == -1)
        {
            perror("sigprocmask");
            exit(1);
        }

        printf("SIGUSR1 is now unblocked.\n");

        sleep(2);

        printf("\nChild process completed.\n");

        exit(0);
    }

    /* Parent process */
    sleep(1);

    printf("\nParent sending SIGUSR1 to child...\n");
```

```
printf("\nParent sending SIGUSR1 to child...\n");

if (kill(pid, SIGUSR1) == -1)
{
    perror("kill");
}
else
{
    printf("SIGUSR1 sent successfully to PID %d\n", pid);
}

sleep(1);

printf("\nProcess Group Information\n");
printf("-----\n");
printf("Parent PID : %d\n", getpid());
printf("Parent PGID : %d\n", getpgid());
printf("Child PID : %d\n", pid);

printf("\nSending SIGUSR1 to child process group...\n");

if (kill(-pid, SIGUSR1) == -1)
{
    perror("kill process group");
}
else
{
    printf("Signal sent to process group %d\n", pid);
}

printf("\nTerminal Signal Test\n");
printf("-----\n");
printf("Press Ctrl+C now to generate SIGINT.\n");
printf("Waiting for 3 seconds...\n");

sleep(3);

printf("\nParent waiting for child completion...\n");

if (waitpid(pid, &status, 0) == -1)
{
    perror("waitpid");
    exit(1);
}

if (WIFEXITED(status))
{
    printf("Child exited normally.\n");
    printf("Exit status: %d\n", WEXITSTATUS(status));

    root@LAPTOP-RJFD8EBT:~# ls -l signal_analysis
-rwxr-xr-x 1 root root 16712 Sep 27 16:14 signal_analysis
root@LAPTOP-RJFD8EBT:~# ./signal_analysis
Signal Delivery and Process Group Demonstration
=====
Parent PID : 1463
Parent PGID : 1463

Child Process
-----
Child PID : 1464
Child PGID : 1464

Blocking SIGUSR1...
SIGUSR1 is now blocked.
Waiting for signal...

Parent sending SIGUSR1 to child...
SIGUSR1 sent successfully to PID 1464

Process Group Information
-----
Parent PID : 1463
Parent PGID : 1463
Child PID : 1464

Sending SIGUSR1 to child process group...
Signal sent to process group 1464

Terminal Signal Test
-----
Press Ctrl+C now to generate SIGINT.
Waiting for 3 seconds...

Unblocking SIGUSR1...

SIGUSR1 received by PID 1464
SIGUSR1 is now unblocked.

Parent waiting for child completion...

Child process completed.
Child exited normally.
Exit status: 0

Signal behavior test completed successfully.
root@LAPTOP-RJFD8EBT:~# |
```

TASK-4: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verifr Execution, Test Background Recovery

```

root@LAPTOP-RJFD8EBT:~# nano job_recovery.c
root@LAPTOP-RJFD8EBT:~# ls -l job_recovery.c
-rw-r--r-- 1 root root 5762 Sep 27 16:20 job_recovery.c
root@LAPTOP-RJFD8EBT:~# cat job_recovery.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[50];
    char status[20];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

/* Add a job to the job table */
void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    job_table[job_count].job_id = job_count + 1;
    job_table[job_count].pid = pid;

    strcpy(job_table[job_count].command, command);
    strcpy(job_table[job_count].status, "Stopped");

    job_count++;
}

/* Update the status of all jobs */
void update_job_status()
{
    int status;
    for (int i = 0; i < job_count; i++)
    {
        pid_t result;
        result = waitpid(
            job_table[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result > 0)
        {
            if (WIFSTOPPED(status))
            {
                strcpy(job_table[i].status, "Stopped");
            }
            else if (WIFCONTINUED(status))
            {
                strcpy(job_table[i].status, "Running");
            }
            else if (WIFEXITED(status))
            {
                strcpy(job_table[i].status, "Completed");
            }
            else if (WIFSIGNALED(status))
            {
                strcpy(job_table[i].status, "Terminated");
            }
        }
    }

    /* Display the job table */
    void list_jobs()
    {
        update_job_status();

        printf("\n");
        printf("Job Table\n");
        printf("===== \n");
        printf("Job ID   PID   Status   Command\n");
        printf("===== \n");

        for (int i = 0; i < job_count; i++)
        {
            printf("%-8d %-8d %-12s %s\n",
                job_table[i].job_id,
                job_table[i].pid,
                job_table[i].status,
                job_table[i].command);
        }
    }
}

/* Create a stopped background job */
void create_stopped_job()
{
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        return;
    }
    if (pid == 0)
    {
        printf("\nChild process started.\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is stopping itself...\n");

        raise(SIGSTOP);

        /* Execution continues after SIGCONT */
        printf("Child resumed successfully.\n");

        printf("Child is running in background...\n");

        sleep(5);

        printf("Child job completed.\n");

        exit(0);
    }

    /* Parent */
    int status;

    /* Wait until child stops */
    waitpid(pid, &status, WUNTRACED);

    add_job(pid, "background recovery job");

    printf("\nStopped Job Created Successfully\n");
    printf("Job ID : %d\n", job_count);
    printf("PID : %d\n", pid);
}

/* Resume a stopped job */
void resume_job(int id)

```

```

-rwxr-xr-x 1 root root 16864 Sep 27 16:20 job_recovery
root@LAPTOP-RJFD8EBT:~# ./job_recovery
Background Job Recovery System
=====

1. Create Stopped Job
2. List Job Table
3. Resume Stopped Job
4. Verify Job Execution
5. Update Job Status
6. Exit
Enter choice: 1

Child process started.
Child PID: 1497
Child is stopping itself...

Stopped Job Created Successfully
Job ID : 1
PID : 1497

1. Create Stopped Job
2. List Job Table
3. Resume Stopped Job
4. Verify Job Execution
5. Update Job Status
6. Exit
Enter choice: 2

Job Table
=====
Job ID   PID   Status   Command
=====
1        1497   Stopped   background recovery job
=====

1. Create Stopped Job
2. List Job Table
3. Resume Stopped Job
4. Verify Job Execution
5. Update Job Status
6. Exit
Enter choice: 3
Enter Job ID: 1

Sending SIGCONT to Job 1...
Continue signal sent successfully.
Child resumed successfully.
Job 1 status updated to Running.

```

```

Default: root@LAPTOP-RJFD8EBT: ~
ctrl+alt+1
Status : Completed
Command : background recovery job

1. Create Stopped Job
2. List Job Table
3. Resume Stopped Job
4. Verify Job Execution
5. Update Job Status
6. Exit
Enter choice: 5
Job status updated.

1. Create Stopped Job
2. List Job Table
3. Resume Stopped Job
4. Verify Job Execution
5. Update Job Status
6. Exit
Enter choice: 2

Job Table
=====
Job ID   PID   Status   Command
=====
1        1497   Completed background recovery job
=====

1. Create Stopped Job
2. List Job Table
3. Resume Stopped Job
4. Verify Job Execution
5. Update Job Status
6. Exit
Enter choice: 4
Enter Job ID: 99
Job 99 not found.

1. Create Stopped Job
2. List Job Table
3. Resume Stopped Job
4. Verify Job Execution
5. Update Job Status
6. Exit
Enter choice: 6
Exiting...
root@LAPTOP-RJFD8EBT:~# |

```